

How **Module Federation** actually works

A shell app draws the frame. Separate apps, built and deployed on their own, ship chunks of finished screen into it while the page is running. Nothing is imported at build time. These eight cards explain the mechanism from the ground up, using the plugins this repository actually builds.

THE ONE-SENTENCE VERSION

Module Federation lets **one JavaScript application load code out of another application at runtime**, over HTTP, while sharing a single copy of the libraries they both use — so the two can be written, built and deployed by different people at different times.

READ THIS FIRST IF THE WORDS ARE NEW

A **chunk** is just a JavaScript file the browser can fetch on demand. A **host** is the app that asks for one. A **remote** is the app that offers one. Everything else on these cards is detail hanging off those three words.

THE EIGHT CARDS

- 01 · The problem — one page, many teams
- 02 · The five words you need
- 03 · Build time — what a plugin produces
- 04 · Runtime — the eight steps
- 05 · Shared singletons — why one Vue
- 06 · The seam — one method, one file
- 07 · A worked example, end to end
- 08 · What federation does not give you

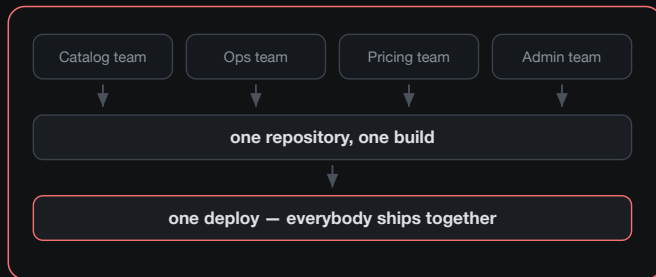
01

WHY ANYBODY WANTED THIS

One page, many teams

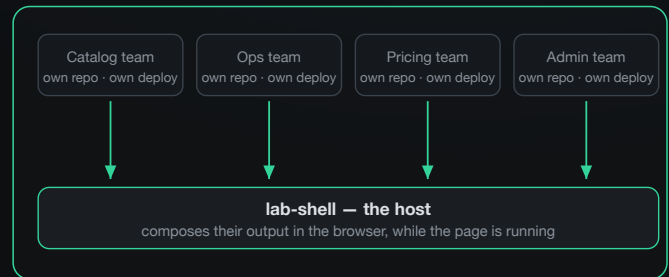
A browser shows one page. A company has many teams. For twenty years those two facts were reconciled by putting every team's code in one build — which means every team ships when the slowest one is ready. Micro-frontends break that link. Module Federation is the plumbing that makes it practical.

THE SINGLE BUILD



Nothing on the right is a build-time link. The host's own build names none of those teams — which is exactly why none of them needs the host to rebuild.

THE FEDERATED PAGE



WHAT THE SINGLE BUILD COSTS

- › One team's failing test blocks four teams' releases
- › Upgrading a shared library is a company-wide event
- › The build gets slower for everyone, forever
- › Ownership blurs: everything is in everyone's repository

WHAT FEDERATION BUYS

- › A team ships when **it** is ready, on its own pipeline
- › Adding a feature to the product is a **configuration change**, not a host rebuild
- › A broken feature can be switched off without redeploying anything
- › The page still feels like one app, because the shell owns the frame

IN THIS REPO

The host is `lab-shell`. The features are the folders under `lab-shell/plugins/` — `demo-catalog` on port 7112, `example-plugin` on 7111, and three deliberately broken ones used to prove failures stay contained. Each has its own `package.json`, its own `vite.config.js` and its own dev server. **The shell imports none of them.**

02

THE FIVE WORDS YOU NEED

Host, remote, container, exposed module, chunk

Every explanation of Module Federation uses these five words in the first paragraph and defines none of them. Here they are, mapped onto two real folders in this repository: the shell at `lab-shell/` and the demo plugin at `lab-shell/plugins/demo-catalog/`.

REMOTE BUILD — DEMO-CATALOG, PORT 7112

remoteEntry.js — the

container a tiny file listing what this plugin offers. Nothing else. **E ./plugin — the exposed module** maps to `src/plugin.js`. The only door in; everything else is private. **J chunk-*.js — the chunks** the actual views and CSS. Fetched only when the plugin is first used. **HOST BUILD — LAB-SHELL, PORT 7110** **H lab-shell — the host** its `vite.config.js` declares `remotes: {}` — an empty list. It is built without knowing any plugin exists. **AT RUNTIME — IN THE BROWSER** `registerRemotes([ { name, entry, type: 'module' } ])` `loadRemote('demo_catalog/plugin')` fetch, on first use The arrow crosses a network boundary, not a build boundary. That is the whole trick.

HOST

The app that asks. Owns the page, the router, the layout. Here: `lab_shell`.

REMOTE

The app that offers. Built and deployed alone. Here: `demo_catalog`, `example_plugin`.

CONTAINER

`remoteEntry.js`. A menu, not a meal — it names what can be loaded and how.

A NAMING
DETAIL
THAT TRIPS
PEOPLE

The container name is snake_case (`demo_catalog`) because it can become a global JavaScript identifier in some output formats. The plugin's id stays kebab-case (`demo-catalog`), and the plugin manifest carries the mapping between the two. They are different names for different layers — do not "tidy" one into the other.

03

WHAT A PLUGIN BUILD PRODUCES

Six lines of config, and what each one makes

A plugin is an ordinary Vite app with one extra build plugin. This is the whole of `lab-shell/plugins/demo-catalog/vite.config.js` that matters, and what each key is responsible for in the finished output.

CONFIG KEY	VALUE, AS BUILT	WHAT IT PRODUCES, AND WHY IT IS SET THAT WAY
name	'demo_catalog'	The container's identity. The host uses this exact string in <code>loadRemote('demo_catalog/plugin')</code> . snake_case, because it can become a global identifier.
filename	'remoteEntry.js'	The file the host fetches first. It is the plugin's public address — here, <code>http://localhost:7112/remoteEntry.js</code> .
exposes	{ './plugin': './src/plugin.js' }	The one door in. Everything not listed here is private to the plugin, no matter what it exports.
shared	vue, @primeuix/styled both singleton: true	Libraries the plugin will borrow from the host instead of shipping its own. See Card 05 — getting this wrong is the classic federation bug.
bundleAllCSS	true	Puts the plugin's styles inside its chunks, so a loaded plugin arrives already styled. No separate stylesheet for the host to know about.
build.target	'esnext'	Modern module output. The host must load this as <code>type: 'module'</code> — see the warning below.

WHAT IS IN THE PLUGIN'S DIST FOLDER

- `remoteEntry.js` — small, always fetched first
- `__federation_expose_plugin-*.js` — the exposed module
- `chunk-*.js` — views, components and CSS, fetched behind it
- `manifest.json` — this repo's own metadata file, **not** a federation file

THE MANIFEST IS NOT FEDERATION

Module Federation knows nothing about routes, navigation entries or panels. Those live in this repo's own `manifest.json` and are read by the shell **before** any federation call happens. Federation only ever answers one question: "give me the code behind this name."

THE FAILURE THAT WASTES AN AFTERNOON

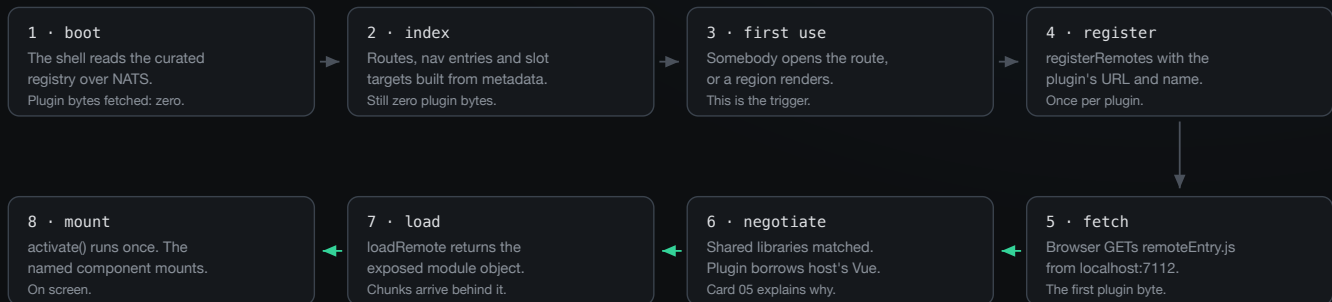
Vite emits ES modules, so the host must register the remote with `type: 'module'`. Federation's default is a classic script, which dies on the first `import` statement with a raw `SyntaxError` naming neither the plugin nor the cause. It looks like a broken bundle. It is one missing word.

04

FROM COLD PAGE TO MOUNTED PANEL

The eight steps, in order

This is the whole mechanism. Steps 1 and 2 happen at boot for every plugin. Steps 3 to 8 happen once per plugin, the first time something on screen actually needs it — and never again.



Two components asking in the same tick share one load and one activate() — the shell keeps a promise per plugin id, so the work is never done twice.

WHY STEPS 1–2 MUST FETCH NOTHING

A booted shell has a complete navigation tree and a complete route table with **zero** plugin chunks fetched. Metadata before code. Twenty plugins cost twenty manifest entries at boot, not twenty bundles — so the shell's start-up time does not grow with the product.

WHAT "FIRST USE" MEANS IN PRACTICE

- › A user clicks a nav entry the plugin contributed
 - › A page renders an extension region the plugin fills
 - › A topbar control or footer item comes into view
- Any of these is enough. None of them happens at boot.

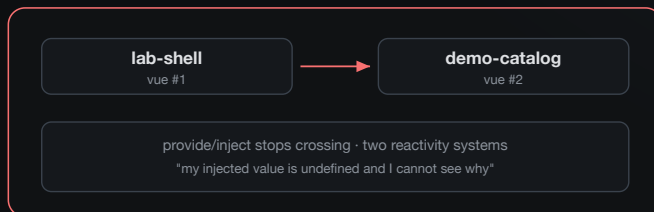
05

THE ONE SETTING THAT BREAKS EVERYTHING

Why there must be exactly one Vue

The host is a Vue app. Every plugin is written in Vue. If each one ships its own copy, the page ends up running two Vues that cannot see each other — and the symptoms look nothing like the cause.

WITHOUT SINGLETON: TRUE



Also shared as a singleton here: @primeuix/styled, so host and plugin resolve to one theme registry rather than two.

WITH SINGLETON: TRUE — AS BUILT



SYMPTOMS OF A DUPLICATED VUE

- › `inject()` returns `undefined` in a plugin component
- › A ref updates but nothing re-renders
- › Theme tokens apply in the shell and not in the panel
- › Console warnings about multiple Vue instances, easily missed

THE TWO SETTINGS THAT PREVENT IT

- › `shared: { vue: { singleton: true, requiredVersion: '^3.5' } }` in **both** configs
 - › `resolve.dedupe: ['vue', 'primevue', '@primeuix/styled']` in the host
- `requiredVersion` is the guard: if a plugin needs a Vue the host cannot supply, federation says so at load time rather than half-working.

THE TRADE
THIS MAKES

Sharing is what makes a plugin small and the page coherent — but it is also a real coupling. A plugin cannot unilaterally jump to a Vue major version the shell does not run. That is the honest cost of federation, and the reason the shared list is kept short and deliberate: Vue and the theme registry, nothing else.

06

KEEPING FEDERATION REPLACEABLE

One method, in one file

Module Federation is a choice, not a law. In this repo the whole of it is confined behind a four-line interface, so the shell above it never learns the word "federation" — and could be moved to import maps, or to plain dynamic import, by editing one file.

shell

pluginLoader.js

curation guard · status machine · one load per plugin id

everything the product cares about
lives above this line

{ async load(remote) → module }

the entire contract between the shell and any loading technology

four words. no URLs, no globals,
no mention of a bundler**federatedAdapter.js**

registerRemotes once · loadRemote · type: 'module'

the only file in the repository
that says "federation"

library

@module-federation/runtime

third-party, replaceable.

WHAT THE ADAPTER ACTUALLY DOES

- › Turns a relative URL into an absolute one against the page
- › Calls `registerRemotes` once per plugin, tracked in a `Set`
- › Strips the leading `./` and calls `loadRemote('name/expose')`
- › Throws a clear error if the remote returns nothing

That is the whole file. Roughly thirty lines.

WHY THIS SEAM IS WORTH HAVING

Federation is young and the ecosystem moves. Import maps are a browser-native alternative that may make this library unnecessary. Because the shell only ever calls `load(remote)`, swapping is a change to **this file plus one line of main.js** — not a rewrite of the plugin system.

FAILURES

ARE NAMED,

NOT RAW

Every way a load can fail is mapped to a stage a human can read: `remote-not-curated` → curation check, `chunk-load-failed` → remote entry fetch, `malformed-module` → module resolution, `activate-threw` → plugin activation, `render-threw` → contribution render. So "plugin X failed" always comes with "at which step".

07

END TO END, ONE REAL PLUGIN

From a line of metadata to a panel on screen

`example-plugin` exists to demonstrate one contribution of every kind. Its `manifest.json` names components as strings; its `plugin.js` exports those same names as real Vue components. The string is the only link between them — and it is checked at load time.

MANIFEST.JSON — METADATA, NO CODE

```
route · component: "overview"
extension · component: "home-panel"
extension · component: "demo-sidebar"
shell-control · component: "topbar-control"
shell-footer · component: "footer-item"
```

PLUGIN.JS — THE EXPOSED MODULE

```
overview: OverviewView
'home-panel': HomePanel
'demo-sidebar': DemoSidebarPanel
'topbar-control': TopbarControl
'footer-item': FooterItem
```

The left column is read at boot. The right column is fetched the first time one of those five places is on screen.

MANIFEST, AT BOOT

Read over NATS from the curated registry. Gives the shell a route, a nav entry and three slot targets — with no code fetched.

MODULE, AT FIRST USE

`loadRemote` returns `{ components, activate }`.
`activate(shellApi)` runs once and hands the plugin a frozen, read-only shell API.

RENDER

The shell looks up the string in `components`. Missing name, or a component that throws, fails that one contribution — not the page.

THE DEMO CASE, IN THIS REPO'S WORDS

`demo-catalog` exposes exactly two components — `catalog` (the demo menu) and `intro` (a demo's intro page). The shell draws the topbar, the rail and the footer; the demo's own screen arrives as a chunk from port 7112 and mounts inside that frame. To the user it is one application. To the build system it is two, and neither one imports the other.

08

WHERE THE LIBRARY STOPS

What Module Federation does not give you

Federation fetches code and shares libraries. That is all it does. Everything a real product needs around that — deciding **which** code may be fetched, containing it when it misbehaves, and proving where it came from — is the shell's own work, and none of it comes in the box.

NOT PROVIDED	WHY IT MATTERS	HOW THIS REPO ANSWERS IT
Trust	Any URL federation is given, it will fetch and run.	A curated registry served over NATS. The loader will only fetch a URL an operator put on the allowlist.
Isolation	A remote's code runs on the host's page, with the host's privileges.	A frozen, read-only shell API is the only surface handed to a plugin. Extension regions are host-owned.
Containment	One bad component can blank a page.	A failing contribution fails alone; the rest of the page renders. Deliberately broken fixtures prove it.
Versioning	Nothing checks that a plugin fits this shell.	Schema and shell-API version checks at validation, before any code is fetched.
Provenance	A fetched bundle is anonymous.	Signing gates — a release that is not owned, is revoked, or is stale is refused at index time.
Capacity	Unlimited contributions into one region wreck a layout.	Regions declare capacity; over-capacity contributions are refused when the index is built, not when it paints.

WHEN NOT TO REACH FOR THIS

- One team owning the whole front end — the coordination cost buys nothing
- A page where every part ships together anyway
- An app small enough that the build is not the bottleneck

Federation is the answer to an organisational problem. Without that problem it is just extra moving parts.

WHEN IT EARNS ITS KEEP

- Several teams with genuinely separate release cadences
- Features that must be switchable without a host deploy
- Third-party or tenant-supplied UI you must curate rather than trust

All three are true here, which is why the lab shell is built this way.

SOURCES

Martin Fowler, *Micro Frontends* — martinfowler.com/articles/micro-frontends.html for the integration approaches and the team-autonomy argument. DevJam, *Micro-Frontend Architecture* — devjam.in/frontend-architect/micro-frontends/architecture for the host-container and remote-mount-slot shape. Both are generic; every concrete name, port and file path on these cards is this repository as built.